

services\analysis\cwe\analyzer.py

```
"""
CWE Risk Analysis Functions
Handles vendor risk analysis, weighted analysis, and 30-day counts
"""

# Efficient counting for CWE frequency analysis
from collections import Counter
# Type hints for function return values
from typing import Dict, Any
# Date operations for temporal analysis
from datetime import datetime
# CWE title resolution utility
from .utils import get_cwe_title

def get_vendor_risk_analysis() -> Dict[str, Any]:
    """Get CWE analysis for 2025 ONLY - MEMORY OPTIMIZED"""
    print("[Vendor Risk Analysis] Analyzing CWE patterns for 2025 only...")
    try:
        from services.data.data_processor import historical_loader

        # ONLY LOAD 2025 - saves memory!
        current_year = datetime.now().year
        years_to_analyze = [current_year] # CHANGED: Only current year!

        cwe_counts = Counter()
        cwe_severity_matrix = {}
        total_cves = 0

        for year in years_to_analyze:
            year_cves = historical_loader.get_year_data(year)
            print(f"[Vendor Risk Analysis] Processing {year}: {len(year_cves)} CVEs")
            total_cves += len(year_cves)

            for cve in year_cves:
                cwe = cve.get('CWE')
                if cwe and cwe.startswith('CWE'):
                    cwe_counts[cwe] += 1
                    if cwe not in cwe_severity_matrix:
                        cwe_severity_matrix[cwe] = Counter()
                    severity = cve.get('Severity', 'UNKNOWN').upper()
                    cwe_severity_matrix[cwe][severity] += 1

            print("[Vendor Risk Analysis] Calculating 30-day CWE counts...")
            cwe_30_day_counts = get_cwe_30_day_counts()

            top_cwes = cwe_counts.most_common(20) if cwe_counts else []
            top_5_cwes = top_cwes[:5]
            top_10_cwes = top_cwes[:10]

            result = {
                'top_5_cwes': {
                    'indices': [cwe for cwe, count in top_5_cwes],
                    'labels': [get_cwe_title(cwe) for cwe, count in top_5_cwes],
                    'values': [count for cwe, count in top_5_cwes]
                },
                'top_10_cwes': {
                    'indices': [cwe for cwe, count in top_10_cwes],
                    'labels': [get_cwe_title(cwe) for cwe, count in top_10_cwes],
                    'values': [count for cwe, count in top_10_cwes]
                }
            }
```

```

    },
    'all_cwes': {
        'indices': [cwe for cwe, count in top_cwes],
        'labels': [get_cwe_title(cwe) for cwe, count in top_cwes],
        'values': [count for cwe, count in top_cwes]
    },
    'severity_matrix': {
        cwe: dict(severity_counts) for cwe, severity_counts in
        cwe_severity_matrix.items()
    },
    'cwe_30_day_counts': cwe_30_day_counts or {},
    'total_cves_analyzed': total_cves,
    'years_analyzed': years_to_analyze
}

print(f"[Vendor Risk Analysis] Analyzed {result['total_cves_analyzed']}
CVEs, found {len(top_cwes)} unique CWEs")
print(f"[Vendor Risk Analysis] Top 5 CWEs: {[cwe for cwe, count in
top_5_cwes]}")

return result

except Exception as e:
print(f"[Vendor Risk Analysis] Error in analysis: {e}")
return {
    'top_5_cwes': {'indices': [], 'labels': [], 'values': []},
    'top_10_cwes': {'indices': [], 'labels': [], 'values': []},
    'all_cwes': {'indices': [], 'labels': [], 'values': []},
    'severity_matrix': {},
    'cwe_30_day_counts': {},
    'total_cves_analyzed': 0,
    'years_analyzed': []
}

def get_cwe_30_day_counts() -> Dict[str, int]:
    """Get CWE counts for last 30 days from API data"""
    try:
        # Get recent vulnerability data for current threat assessment
        from services.data.api_client import api_client
        api_cves = api_client.get_cves_last_30_days()

        # Count CWE frequency in recent data
        cwe_30_day_counts = Counter()
        for cve in api_cves:
            cwe = cve.get('CWE')

            # Validate CWE format before counting
            if cwe and cwe.startswith('CWE'):
                cwe_30_day_counts[cwe] += 1

        print(f"[Vendor Risk Analysis] Found {len(cwe_30_day_counts)} unique CWEs
        in last 30 days")

        # Convert to standard dict for JSON compatibility
        return dict(cwe_30_day_counts)

    except Exception as e:
        # Return empty dict to allow continued operation when API unavailable
        print(f"[Vendor Risk Analysis] Error getting 30-day CWE counts: {e}")
        return {}

```

```

def get_weighted_cwe_analysis() -> Dict[str, Any]:
    """Get weighted CWE analysis (severity-weighted scores) - LAZY LOADING"""
    print("[Vendor Risk Analysis] Calculating weighted CWE scores...")

    try:
        # Use same temporal scope as vendor risk analysis for consistency
        current_year = datetime.now().year
        years_to_analyze = [current_year - 1, current_year]

        # Load historical data for weighted analysis - ONE YEAR AT A TIME
        from services.data.data_processor import historical_loader

        # Define severity weights for risk scoring (higher = more critical)
        severity_weights = {
            'CRITICAL': 4, # Highest priority
            'HIGH': 3,     # High priority
            'MEDIUM': 2,   # Medium priority
            'LOW': 1,      # Low priority
            'UNKNOWN': 1   # Default weight for unclassified
        }

        # Calculate weighted scores for each CWE
        cwe_weighted_scores = Counter()

        for year in years_to_analyze:
            year_cves = historical_loader.get_year_data(year) # Lazy load

            for cve in year_cves:
                cwe = cve.get('CWE')

                # Process valid CWE identifiers
                if cwe and cwe.startswith('CWE'):
                    severity = cve.get('Severity', 'UNKNOWN').upper()

                    # Apply severity weight to CWE score
                    weight = severity_weights.get(severity, 1)
                    cwe_weighted_scores[cwe] += weight

        # Get top 10 highest weighted CWEs for strategic focus
        top_weighted = cwe_weighted_scores.most_common(10) if cwe_weighted_scores
        else []

        # Structure result for visualization and analysis
        result = {
            'indices': [cwe for cwe, score in top_weighted],
            'labels': [get_cwe_title(cwe) for cwe, score in top_weighted],
            'values': [score for cwe, score in top_weighted]
        }

        print(f"[Vendor Risk Analysis] Weighted analysis complete: {len(top_weighted)} CWEs")
        return result

    except Exception as e:
        # Return empty structure for graceful error handling
        print(f"[Vendor Risk Analysis] Error in weighted analysis: {e}")
        return {'indices': [], 'labels': [], 'values': []}

```